Welcome to Slidev

Presentation slides for developers

Press Space for next page →

What is Slidev?

Slidev is a slides maker and presenter designed for developers, consist of the following features

- 📝 **Text-based** - focus on the content with Markdown, and then style them later
- 🎨 **Themable** - theme can be shared and used with npm packages
- 🧑‍💻 **Developer Friendly** - code highlighting, live coding with autocompletion
- 🤹 **Interactive** - embedding Vue components to enhance your expressions
- 🎥 **Recording** - built-in recording and camera view
- 📤 **Portable** - export into PDF, PNGs, or even a hostable SPA
- 🛠️ **Hackable** - anything possible on a webpage

Read more about Why Slidev?

agenda

1. Welcome to Kubernetes
2. What is Slidev?
3. agenda
4. kubernetes
5. kubernetes-apm
6. kubernetes-authentication-authorization
7. kubernetes-capacity-planning
8. kubernetes-certificates
9. kubernetes-chaos-engineering
10. kubernetes-concepts
11. kubernetes-configuration-management
12. kubernetes-configurations
13. kubernetes-container-security
14. kubernetes-cronjob
15. kubernetes-daemonset
16. kubernetes-deployment
17. kubernetes-devops
18. kubernetes-failures
19. kubernetes-finops
20. kubernetes-policy-enforcement
21. kubernetes-pods
22. kubernetes-platform-engineering
23. kubernetes-operator
24. kubernetes-sosivio
25. kubernetes-signoz
26. kubernetes-opentelemetry
27. kubernetes-namespaces
28. kubernetes-multi-cloud
29. kubernetes-manifests
30. kubernetes-logging-monitoring
31. kubernetes-learning-roadmap
32. kubernetes-kiali
33. kubernetes-job
34. kubernetes-istio
35. kubernetes-ingress
36. kubernetes-gitops
37. kubernetes-gateway
38. kubernetes-production-best-practices
39. kubernetes-prometheus
40. kubernetes-registry
41. kubernetes-replicaset
42. kubernetes-resourcequota
43. kubernetes-scheduling
44. kubernetes-secrets
45. kubernetes-security
46. kubernetes-service
47. kubernetes-service-mesh
48. kubernetes-sre
49. kubernetes-statefulset
50. kubernetes-storage
51. kubernetes-templating
52. kubernetes-vault-secrets
53. kubernetes-vulnerability
54. kubernetes-vulnerability-scanning

kubernetes

microk8s install

```
sudo snap install microk8s --classic
microk8s status --wait-ready
```

microk8s enable addons

```
microk8s enable dashboard dns registry istio
```

```
microk8s kubectl get namespaces
NAME                 STATUS   AGE
argo-events          Active   2d21h
argo-rollouts        Active   2d22h
argocd               Active   2d22h
container-registry   Active   2d22h
default              Active   2d22h
kube-node-lease      Active   2d22h
kube-public          Active   2d22h
kube-system          Active   2d22h
```

microk8s dashboard

```
microk8s dashboard-proxy
Dashboard will be available at https://127.0.0.1:10443
Use the following token to login:
eyJhbGciOiJSUzI1NiIsImtpZCI6Im1nSkxLVmpFMnN6Mk1QSDNBVFBJUDh6QXVWYk5VZTllSXZ1N256OUtrcUkifQ.eyJpc3MiOiJrdWJlcm5ldGVzL3NlcnZpY2VhY2NvdW50Iiwia3ViZXJuZXRlcy5pby9zZXJ2aWNlYWNjb3VudC9uYW1lc3BhY2UiOiJrdWJlLXN5c3RlbSIsImt1YmVybmV0ZXMuaW8vc2VydmljZWFjY291bnQvc2Vjcm
```

troubleshooting

```
kubectl get pods --all-namespaces
NAMESPACE            NAME                                         READY   STATUS             RESTARTS          AGE
kube-system          coredns-7896dbf49-xrdl6                      0/1     ImagePullBackOff   0                 3d22h
kube-system          dashboard-metrics-scraper-6b96ff7878-dd8nw   0/1     ImagePullBackOff   0                 3d22h
kube-system          hostpath-provisioner-5fbc49d86c-mhrrw        0/1     ErrImagePull       0                 3d22h
kube-system          kubernetes-dashboard-7d869bcd96-dpsb4        0/1     ImagePullBackOff   0                 3d22h
```

```
kubectl describe pod kubernetes-dashboard-7d869bcd96-dpsb4 -n kube-system
sudo microk8s.ctr images pull .../kubernetesui/dashboard:v2.7.0
sudo microk8s.ctr images tag .../kubernetesui/dashboard:v2.7.0 kubernetesui/dashboard:v2.7.0
sudo microk8s.ctr images ls
```

```
systemctl daemon-reload
systemctl restart kubelet
sudo snap restart microk8s
```

kubernetes-apm

Performance optimization, Efficient resource usage, Proactive issue detection, Rapid troubleshooting and debugging, Capacity planning and scalability, Enhanced security and compliance and etc.

Cluster-level monitoring:

- Node functions
- Node availability
- Node resource usage
- Number of pods running

Pod-level monitoring

- Kubernetes metrics
  - Number of pod instances
  - Pod status, restarts
  - CPU, Memory usage
  - Network usage
- Container metrics
  - CPU,Memory usage/throttling
  - Network traffic/errors
- Application metrics

metrics-server

- kube-controller-manager
- kube-proxy
- kube-apiserver
- kube-scheduler
- kubelet

SigNoz

- Application Performance Monitoring
- Log Management
- Distributed Tracing
- Alerts
- Metrics & Dashboards
- Exceptions

kubernetes-authentication-authorization

kubernetes-capacity-planning

kubernetes-certificates

kubernetes-chaos-engineering

Benefits of Chaos Engineering

- Identify vulnerabilities and weaknesses before they become critical issues:

  Chaos engineering helps uncover hidden flaws in the system that might not be apparent during regular operations. By proactively identifying these vulnerabilities, teams can address them before they lead to significant problems.
- Improve the reliability and resilience of their systems:

  Regular chaos experiments ensure that systems are robust and can handle unexpected disruptions. This continuous testing and improvement process enhances overall system reliability.
- Reduce downtime and improve overall system availability:

  By preparing for potential failures, organizations can minimize the impact of disruptions, leading to reduced downtime and higher availability of services.
- Enhance their ability to respond to failures and outages:

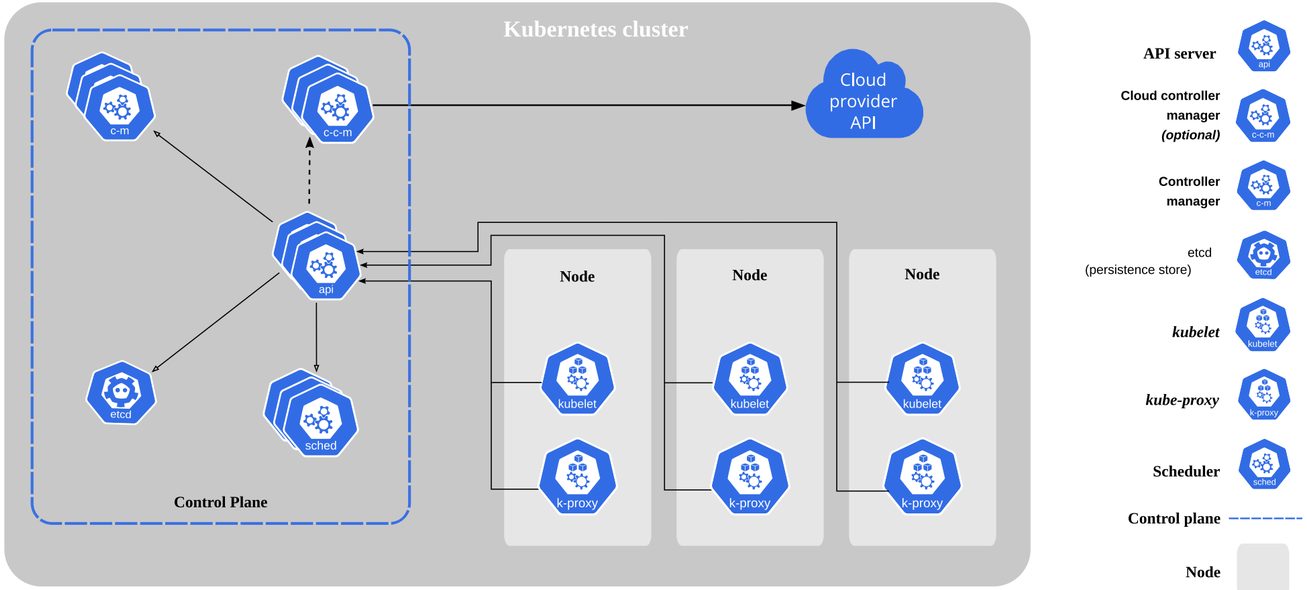
  Chaos engineering equips teams with the knowledge and experience to respond swiftly and effectively to real-world incidents, reducing recovery times and mitigating damage.
- Improve their understanding of how their systems behave under stress:

  Observing system behavior during chaos experiments provides valuable insights into performance bottlenecks and areas for optimization.
- Reduce the risk of cascading failures and improve overall system stability:

  By identifying and addressing weak points, chaos engineering helps prevent small issues from escalating into larger, more severe problems, thereby enhancing system stability.

kubernetes-concepts

- **Service discovery and load balancing** Kubernetes can expose a container using the DNS name or using their own IP address.
- **Storage orchestration** Kubernetes allows you to automatically mount a storage system of your choice, such as local storages, public cloud providers, and more.
- **Automated rollouts and rollbacks** You can describe the desired state for your deployed containers using Kubernetes, and it can change the actual state to the desired state and rate.
- **Automatic bin packing** You provide Kubernetes with a cluster of nodes that it can use to run containerized tasks. You tell Kubernetes how much CPU and memory each container needs.
- **Self-healing** Kubernetes restarts containers that fail, replaces containers, kills containers that don't respond to user-defined health check, doesn't advertise to clients until ready to serve.
- **Secret and configuration management** Kubernetes lets you store and manage sensitive information, such as passwords, OAuth tokens, and SSH keys.
- **Batch execution** In addition to services, Kubernetes can manage your batch and CI workloads, replacing containers that fail, if desired.
- **Horizontal scaling** Scale your application up and down with a simple command, with a UI, or automatically based on CPU usage.
- **IPv4/IPv6 dual-stack** Allocation of IPv4 and IPv6 addresses to Pods and Services
- **Designed for extensibility** Add features to your Kubernetes cluster without changing upstream source code.



kubernetes-configuration-management

kubernetes-configurations

kubernetes-container-security

kubernetes-cronjob

kubernetes-daemonset

kubernetes-deployment

kubernetes-devops

kubernetes-failures

kubernetes-finops

kubernetes-policy-enforcement

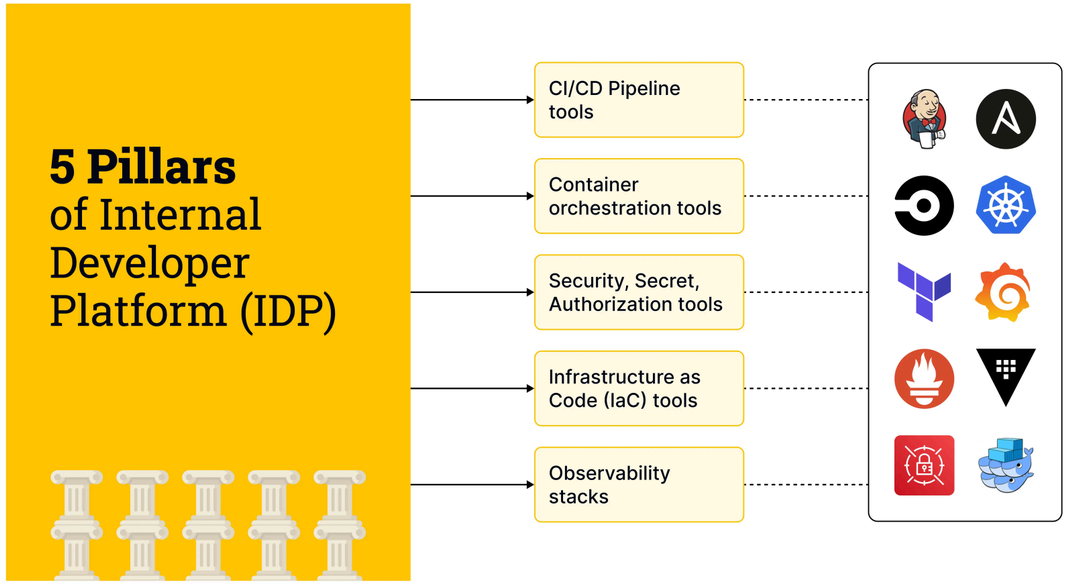
kubernetes-pods

kubernetes-platform-engineering

Challenges of Managing Diverse Applications
In an environment with diverse applications and services, achieving standardization can be elusive. `Platform engineers` need to strike a balance between accommodating these unique requirements and establishing standardized processes that ensure consistency and manageability.

A Paradigm Shift in Platform Engineering
Kubernetes embodies the principles of high availability and fault tolerance, critical aspects of platform engineering. `Platform engineers` can architect systems that maintain continuous service even in the face of unforeseen challenges.

The daunting task of updating applications while minimizing downtime and compatibility hiccups finds a streamlined approach in Kubernetes. `platform engineers` can orchestrate updates that are seamless, incremental, and reversible.



Understanding Multitenancy

- Resource Isolation: Namespaces provide an isolated space where resources such as pods, services, and configurations are contained.
- Security and Access Control: Namespaces allow `platform engineers` to set Role-Based Access Control rules specific to each Namespace.
- Naming Scope: Namespaces ensure naming uniqueness across different teams or projects.
- Logical Partitioning: Platform engineers can logically partition applications.

Kubernetes Controllers: The Automation Engine

- Scaling: HPA automatically adjusts the pod replicas on observed CPU or custom metrics.
- Self-Healing: Liveness and readiness probes contribute to application self-healing.
- Updating: Kubernetes controllers, automate application updates.

Modern Software Development

Agile and Scalable Infrastructure

Streamlined Development Workflows

Standardization and Reusability

Security and Compliance

Monitoring, Observability, and Performance Optimization

kubernetes-operator

kubernetes-sosivio

kubernetes-signoz

Install

```
helm repo add signoz https://charts.signoz.io
kubectl create ns platform
helm --namespace platform install signoz-platform signoz/signoz
- frontend version: '0.59.0'
- query-service version: '0.59.0'
- alertmanager version: '0.23.7'
- otel-collector version: '0.111.9'
- otel-collector-metrics version: '0.111.9'
kubectl -n platform get pods
```

```
export POD_NAME=$(kubectl get pods --namespace platform -l "app.kubernetes.io/name=signoz,app.kubernetes.io/instance=signoz-plat
kubectl --namespace platform port-forward $POD_NAME 3301:3301
```

Pretexting

Synonym of word excuse, but a false one Contact via phone call or other direct ways Attacker try to persuade the victim Some info is already known by the attacker

Baiting

From word bait, but also in cyber context Physical media with malware is used as bait Attacker leaves the bait in a public place Exploit curiosity and greed of people

Kubernetes Infra Metrics and Logs Collection

- Tails and parses logs generated by containers in Kubernetes cluster and sends to SigNoz
- Collects kubelet metrics and host metrics from each nodes of the Kubernetes cluster
- Collects cluster-level metrics from the Kubernetes API server
- Acts as a gateway to send any incoming OTLP telemetry data to SigNoz OtelCollector

override-values.yaml

```
global:
  cloud: others
  clusterName: <CLUSTER_NAME>
  deploymentEnvironment: <DEPLOYMENT_ENVIRONMENT>
otelCollectorEndpoint: ingest.{region}.signoz.cloud:443
otelInsecure: false
signozApiKey: <SIGNOZ_INGESTION_KEY>
presets:
  otlpExporter:
    enabled: true
  loggingExporter:
    enabled: false
```

```
helm install signoz-kubernetes-infra-platform signoz/k8s-infra -f override-values.yaml
```

kubernetes-opentelemetry

kubernetes-namespaces

kubernetes-multi-cloud

kubernetes-manifests

kubernetes-logging-monitoring

kubernetes-learning-roadmap

kubernetes-kiali

kubernetes-job

kubernetes-istio

> Choosing between sidecar and ambient

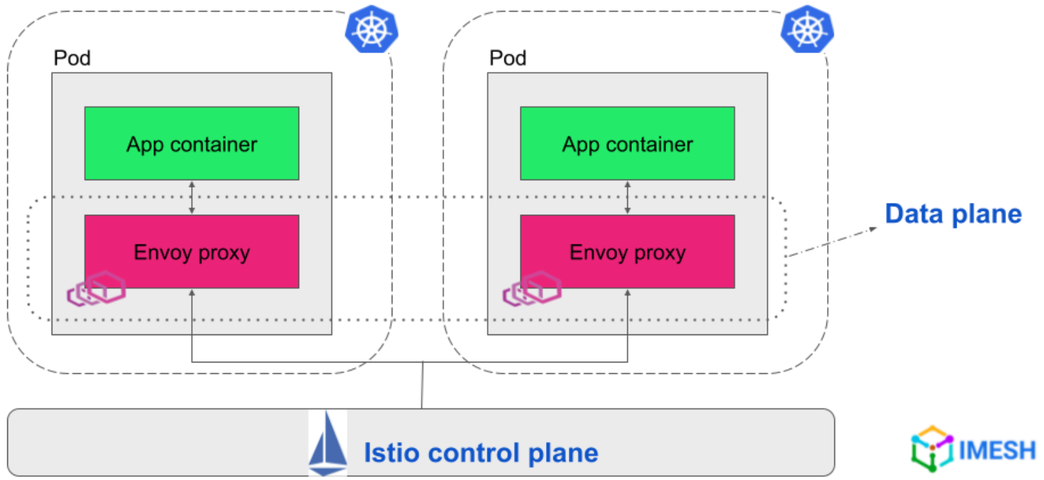
Best Practices:

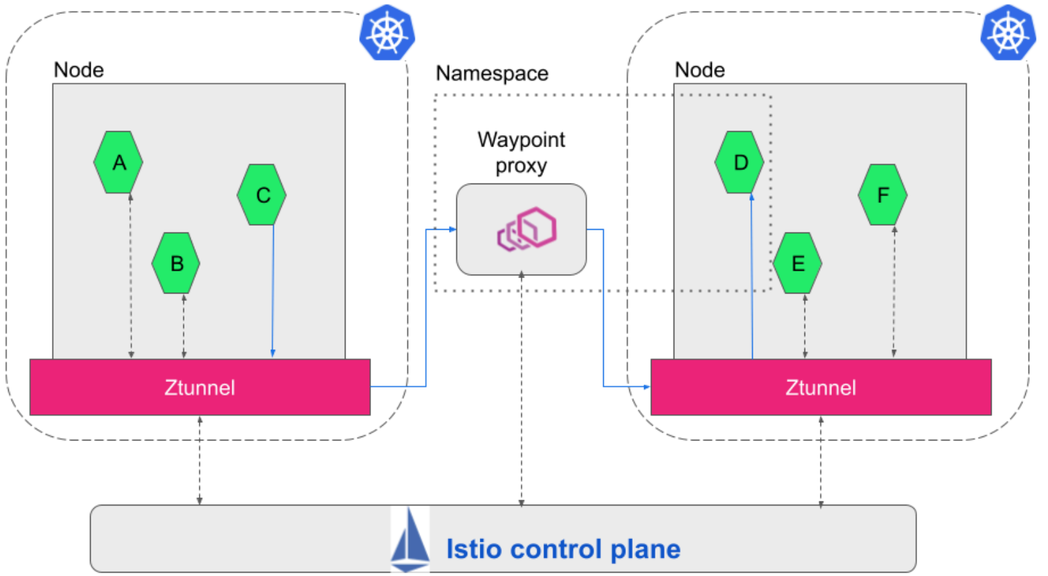
Deployment	Traffic Management	Security	Observability	Image Signing and Validation

Istio supports two main data plane modes:

> The data plane is the set of proxies that mediate and control all network communication between microservices. They also collect and report telemetry on all mesh traffic. The control plane manages and configures the proxies in the data plane.

- **sidecar mode**, which deploys an Envoy proxy along with each pod that you start in your cluster, or running alongside services running on VMs.
- **ambient mode**, which uses a per-node Layer 4 proxy, and optionally a per-namespace Envoy proxy for Layer 7 features.





kubernetes-ingress

kubernetes-gitops

kubernetes-gateway

kubernetes-production-best-practices

Application development

- Health checks
  - Containers have Readiness Liveness probes
  - Containers crash when there's a fatal error
- Apps are independent
  - The Readiness probes are independent
  - The app retries connecting to dependent services
- Graceful shutdown , Fault tolerance
- Resources utilisation, Tagging resources
- Logging
- Scaling
- Configuration and secrets

Governance

- Namespace limits - LimitRange,ResourceQuotas
- Pod security policies
- Network policies
- Role-Based Access Control (RBAC) policies
- Custom policies
- Container Security
- Immutable image, configurations
- Identity-based secrets management

Cluster configuration

- Approved Kubernetes configuration
- Authentication
- Role-Based Access Control (RBAC) - ServiceAccount tokens are for applications, controllers
- Logging setup

Production considerations

- Networking, Security, Complexity
- Availability, Scale, Security
- Serverless, Managed control plane, Managed worker nodes, Integration
- Infrastructure as Code (IaC)
- Monitoring and Centralized Logging
- Centralized Ingress Controller with SSL Certificate Management
- GitOps Deployments
- Secret Management
- Backup and Disaster Recovery
- Storage Scalability and Reliability

kubernetes-prometheus

kubernetes-registry

kubernetes-replicaset

kubernetes-resourcequota

Namespace

- Memory Requests and Limits for a Namespace
- CPU Requests and Limits for a Namespace
- Minimum and Maximum Memory Constraints for a Namespace
- Memory and CPU Quotas for a Namespace
- Pod Quota for a Namespace
- Quotas for API Objects

Pod

API Objects

Storage Resource

kubernetes-scheduling

kubernetes-secrets

kubernetes-security

kubernetes-service

kubernetes-service-mesh

kubernetes-sre

kubernetes-statefulset

kubernetes-storage

kubernetes-templating

kubernetes-vault-secrets

kubernetes-vulnerability

kubernetes-vulnerability-scanning

Why Is Kubernetes Vulnerability Scanning Important?

- Build Scanning - eliminating the chance of the image vulnerability
- Registry Scanning - usually using a built-in image scanner.
- Pre-Deployment Scanning - Open Policy Agent can determine if pods are ready to deploy.
- Runtime Workload Scanning - provide security insights via a unified interface.

Open Source Kubernetes Vulnerability Scanners

- kube-score
- kubeaudit
- Kube-Scan
- Kubesec
- Calico
- Clair
- Trivy
- Anchore

Kubernetes Security Cheat Sheet

- Receive Alerts for Kubernetes Updates
- Securing Kubernetes hosts
- Securing Kubernetes components
- Using the Kubernetes dashboard
- Kubernetes Security Best Practices: Build Phase
- Kubernetes Security Best Practices: Deploy Phase
- Kubernetes Security Best Practices: Runtime Phase

trivy

```
Scanning Commands
  config      Scan config files for misconfigurations
  filesystem  Scan local filesystem
  image       Scan a container image
  kubernetes  [EXPERIMENTAL] Scan kubernetes cluster
  repository  Scan a repository
  rootfs      Scan rootfs
  sbom        Scan SBOM for vulnerabilities and licenses
  vm          [EXPERIMENTAL] Scan a virtual machine image
```

```
FROM openjdk:17-jdk-alpine
ADD ./target/offers-services.jar offers-services.jar
ENTRYPOINT ["java","-jar","/offers-services.jar"]
```

```
trivy image openjdk:17-jdk-alpine
```

```
trivy k8s --report summary microk8s
```

kubernetes-trivy-scanning

> Find vulnerabilities, misconfigurations, secrets, SBOM in containers, Kubernetes, code repositories, clouds and more

```
trivy k8s --report summary microk8s
```